

Graph Preconditioning for High-Dimensional Fixed Effects Regression

Alexander Fischer¹ and Kristof Schröder²

Draft: August 8, 2026

Abstract. The Method of Alternating Projections (MAP) is the standard algorithm for estimating high-dimensional fixed-effect regressions. It is fast when the fixed-effect dimensions are well connected, but converges slowly on sparse or nearly nested fixed-effect graphs, such as matched employer-employee panels where worker-firm mobility links separate worker and firm effects. The convergence behavior of MAP depends on the mobility pattern linking workers to firms, yet MAP only indirectly makes use of this information by iterating over one fixed effect at a time. That mobility pattern is, however, directly encoded in the matrix of worker-firm match counts, which together with the worker and firm count diagonals forms a graph Laplacian - the standard matrix representation of a weighted graph - after a sign flip. This structure lets us approximate its inverse using sparse matrices. We propose a graph-preconditioned iterative solver whose reusable preconditioner is built from small, local factor-pair subproblems - worker-firm, worker-year, and so on - that use the graph directly. Benchmarks show that all methods are fast on well-connected designs. As connectivity weakens, MAP and diagonally preconditioned LSMR slow down, while factor-pair preconditioning remains fast and its runtime falls because the graph preconditioner is cheaper to construct on sparser graphs.

Keywords: high-dimensional fixed effects; alternating projections; preconditioning; matched employer-employee data; computational econometrics

JEL codes: C55; C63; C81; C87; J31

1. Introduction

Fixed effects appear in roughly half of the research published in leading economics and finance journals, according to Goldsmith-Pinkham (2026). Labor economists use worker and firm fixed effects to separate worker heterogeneity from firm wage premia. Health economists study physician practice styles with individual, physician, and region fixed effects in mover designs. Education researchers use school, student, teacher, and student-teacher effects.

The Frisch-Waugh-Lovell (FWL) theorem reduces estimation to two operations: remove the variation explained by the fixed effects from the outcome and regressors, then run a low-dimensional regression on the residualized variables (Frisch and Waugh 1933; Lovell 1963). With high-dimensional fixed effects, software usually performs the first operation with the Method of Alternating Projections (MAP), also known as iterative demeaning or the “Zig-Zag” algorithm (Guimaraes and Portugal 2010; Gaure 2013).

¹trivago

²appliedAI Institute for Europe gGmbH

Leading packages such as Stata’s `reghdfe` (Correia 2014; 2017), R’s `fixest` (Bergé et al. 2026), and Python’s `PyFixest` (PyFixest Developers 2026) use MAP or accelerated variants of it.

MAP handles the fixed-effect dimensions one at a time. It never uses the observed worker-firm links directly. In the worker-firm wage model of Abowd et al. (1999), extended here with year effects, MAP subtracts worker means, then firm means, then year means. Each update uses only the residual left by the previous update; MAP repeats the sequence until changes in the residuals fall below a chosen tolerance. Yet those links matter for identification, precision, and computation (Correia 2017; Jochmans and Weidner 2019). They form a graph in which movers create paths between firms, while stayers add observations without connecting firms. When few workers move between groups of firms, some combinations of worker and firm effects become difficult to separate. MAP can then require many repetitions before estimates in one group reflect changes in another. The same mobility links that identify worker and firm effects thus govern how quickly MAP converges.

Following Jochmans and Weidner (2019), we measure the connectivity of a connected factor-pair graph by λ_2 , the second-smallest eigenvalue of its normalized Laplacian. The graph is weighted by the number of observed co-occurrences. Smaller values indicate weaker connectivity. The measure describes the connectivity of two fixed-effect dimensions. Unless stated otherwise, λ_2 refers to the worker-firm graph. The same graph appears in the weighted cross-product matrix of the fixed-effect indicators, which we call the Gramian (Correia 2017). Its diagonal blocks count observations for each worker, firm, or other fixed-effect level. Its off-diagonal blocks count which levels are observed together, such as the number of observations for each worker-firm pair. MAP uses the diagonal blocks one at a time and does not use these pairwise counts directly.

We therefore build a factor-pair graph preconditioner for designs with a small pairwise λ_2 , where MAP tends to converge slowly. A preconditioner transforms a linear system so that an iterative solver reaches the same solution in fewer steps. Rather than handling the fixed-effect dimensions one at a time, our preconditioner builds a local problem for every pair, such as worker-firm and worker-year, and uses the observed links directly. After a sign change, each pair block is a graph Laplacian, the standard matrix representation of a weighted graph. Sparse matrix methods can approximate its inverse at low cost (Spielman and Teng 2014; Gao et al. 2025). We use a weighted sum of these approximate pair solutions as the preconditioner for LSMR.³

Constructing the factor-pair preconditioner takes time. On a well-connected graph, the reduction in iterations may not repay this setup cost because MAP and diagonal preconditioning are already fast. As λ_2 falls, however, the reduction in iterations can outweigh the construction cost. Figure 1 compares total regression times as worker-firm connectivity varies. The top row changes worker mobility; the bottom row changes sorting among movers. In each row, the left panel compares package defaults and the right panel compares solvers within PyFixest. We choose each solver’s tolerance so that coefficient and residual errors are similar. Section 7 compares run time at the accuracy each method actually achieves.

³`FixedEffectModels.jl` (FixedEffectModels.jl Developers 2026) also uses LSMR (Fong and Saunders 2011). It applies diagonal preconditioning, which uses fixed-effect counts but not the links between fixed-effect dimensions. We instead precondition LSMR with the factor-pair graph.

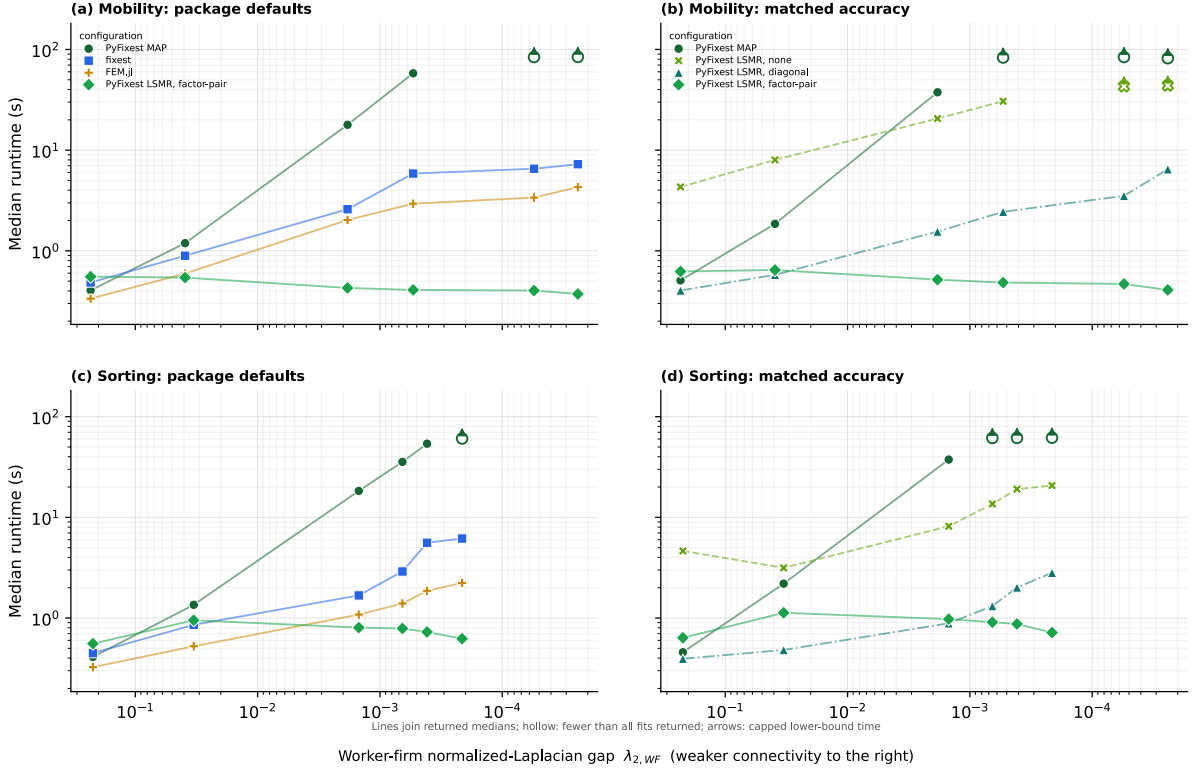


Figure 1: The panels plot median regression time against the worker-firm normalized-Laplacian gap $\lambda_{2,WF}$ on logarithmic axes. We compute λ_2 after removing singleton fixed-effect levels. Smaller values mean weaker connectivity. All panels use the same reversed horizontal scale, so λ_2 decreases from left to right. Each simulated worker-firm-year panel has 1 million observations. The top panels vary worker mobility; the bottom panels hold move probability at one and vary sorting. The left panels compare packages at their default settings, including their own treatment of fixed-effect levels observed only once. The right panels compare four PyFixest solver configurations, with tolerances chosen to give similar coefficient and residual errors. Filled markers indicate that all three planned fits produced estimates; hollow markers indicate that only one or two did. Lines join the medians in order of λ_2 and are not fitted trends. An arrow marks the median time for fits that reach the iteration limit. Because these fits did not finish, the marked time is a lower bound. Arrows are not joined to the lines. Fits that end in another error are omitted.

At high connectivity, all implementations finish quickly. As λ_2 falls, MAP and LSMR without factor-pair preconditioning slow down. Factor-pair LSMR remains fast. In the lowest-mobility designs, its run time falls because the worker-firm subproblems are cheaper to construct.

Sections 2-5 set up fixed-effect absorption and connect MAP convergence to graph connectivity. Section 6 develops the factor-pair preconditioner, and Section 7 reports the benchmarks. Section 8 describes the software; Section 9 concludes. The appendices give the algorithm, report additional benchmarks, and compare coefficient estimates and memory use.

2. Absorbing Fixed Effects⁴

We study the linear model

$$y = X\beta + D\alpha + \varepsilon, \quad (1)$$

⁴Researchers employ several names for this operation: “absorbing fixed effects”, “demeaning”, “residualizing”, or applying the “within transformation”. We use these terms interchangeably throughout.

where X contains the regressors of interest, D is the fixed-effect design matrix, and α collects the fixed-effect coefficients. In high-dimensional applications, D may have hundreds of thousands or millions of columns. Forming $[X \ D]$ or inverting its cross product can then be computationally infeasible. The Frisch-Waugh-Lovell (FWL) theorem lets us compute $\hat{\beta}$ without forming $[X \ D]$ or inverting its cross product.

FWL reduces the computation of $\hat{\beta}$ to two steps. First, we regress y on D and retain the residual $\tilde{y}$. We repeat this operation for every column of X . Let M_D denote the linear operator that maps a variable to its residual, so that $\tilde{y} = M_D y$ and $\tilde{X} = M_D X$. In the second step, we regress the residualized outcome on the residualized covariates,

$$\tilde{y} = M_D y, \quad \tilde{X} = M_D X, \quad \hat{\beta} = (\tilde{X}' W \tilde{X})^{-1} \tilde{X}' W \tilde{y}, \quad (2)$$

where W is a diagonal matrix of weights. With one covariate, this amounts to three regressions: y on D , x on D , and $\tilde{y}$ on $\tilde{x}$. FWL implies that the slope in the last regression equals the coefficient on x in the full regression of y on x and D .

To residualize the outcome and each covariate in X , we project them onto the column space of the fixed-effect dummy matrix D by weighted least squares. For a variable μ to be residualized - either y or one column of X - we solve

$$\hat{\alpha}_\mu = \arg \min_{\alpha} \| D\alpha - \mu \|_W^2, \quad (3)$$

and the residual is $\tilde{\mu} = \mu - D\hat{\alpha}_\mu$. The first-order condition for Equation 3,

$$D'W(D\hat{\alpha}_\mu - \mu) = 0, \quad \text{equivalently} \quad G\hat{\alpha}_\mu = D'W\mu, \quad G = D'WD, \quad (4)$$

determines $\hat{\alpha}_\mu$. The coefficient matrix G is the same for each FWL residualization; only the variable being residualized changes. The cost of residualization therefore depends on the structure of the Gramian G , the weighted cross-product matrix of the fixed-effect dummies. The AKM worker-firm model gives this matrix a concrete interpretation. Worker moves link firms through shared workers and determine the off-diagonal blocks of G .

3. A Running Example: The AKM Model

The AKM model of Abowd et al. (1999) provides the worker-firm setting used throughout the paper. It uses workers who move across firms to separate persistent worker heterogeneity from firm wage premia. We write the AKM regression equation as

$$y_{it} = \alpha_i + \psi_{J(i,t)} + \varphi_t + x'_{it}\beta + \varepsilon_{it}, \quad (5)$$

where α_i is a worker fixed effect, $\psi_{J(i,t)}$ is the fixed effect for the firm employing worker i at time t , and φ_t is a time fixed effect.

The AKM specification maps naturally to a graph. Workers and firms are nodes in a bipartite graph; each employment spell contributes an edge. Worker moves induce a firm-to-firm graph: two firms are connected when at least one worker is observed at both firms. Stayers - workers who never change their employer - add observations to existing worker-firm links but do not create bridges between firms.

Year effects enter as a third, low-dimensional factor observed on the same worker-firm records. Figure 2 contrasts a well-connected mobility graph with one that fragments under strong sorting.

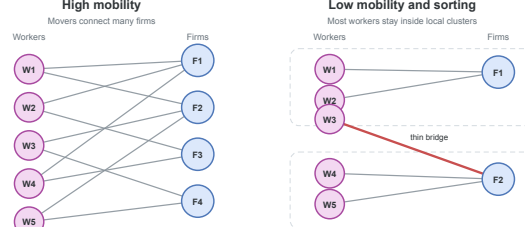


Figure 2: Worker-firm graphs under high and low mobility. High mobility creates many paths between firms. With low mobility and strong sorting, only a few worker moves connect otherwise separate groups of firms.

These mobility links matter for identification and computation. In AKM, movers provide the comparisons that separately identify worker and firm fixed effects by distinguishing worker heterogeneity from firm wage premia. A worker observed at only one firm provides no such comparison: a high wage could reflect an unusually productive worker, a high-wage firm, or both. Without worker moves, these components are not separately identified. Movers observed at firms with different wage profiles help attribute wage variation to worker effects or firm premia. If a worker earns high wages across several firms, the comparison points toward a worker effect. If many different workers earn higher wages at the same firm, it points toward a firm premium. The more such cross-firm comparisons the data contain, especially across otherwise different firms, the easier it is to identify worker and firm premia. In the graph, these moves are exactly the edges that connect firms. In the Gramian, the same links appear in the worker-firm cross-tabulation.

4. The Graph Structure of the Gramian

The block structure of the Gramian $G = D'WD$ represents the bipartite graph of worker and firm connections introduced in Section 3 (Correia 2017). Suppose that the columns of D are ordered as worker levels, firm levels, and year levels. Then

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}. \quad (6)$$

The **diagonal blocks** G_{WW} , G_{FF} , and G_{YY} contain weighted counts for workers, firms, and years. An observation belongs to one level of each factor, so these blocks are diagonal; solving them requires only division by group counts.

The **off-diagonal blocks** are cross-tabulations: the worker-firm block C_{WF} records how often worker i is observed at firm j , and the worker-year and firm-year blocks have analogous interpretations.

The following small worker-firm panel illustrates how to populate its Gramian. We ignore regression weights and set $W = I$.

Obs.	Worker	Firm	Year	y
1	W_1	F_1	Y_1	3.2
2	W_1	F_2	Y_2	4.1
3	W_2	F_1	Y_1	2.8

Obs.	Worker	Firm	Year	y
4	W_2	F_1	Y_2	3.9
5	W_3	F_2	Y_1	5.0
6	W_3	F_2	Y_2	4.5

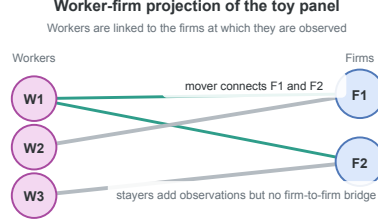


Figure 3: Worker-firm graph for the example panel. Worker W_1 works at both firms and connects them. Worker W_2 works only at F_1 , and worker W_3 works only at F_2 .

Figure 3 plots the worker-firm projection of this panel. Worker W_1 has employment spells at both F_1 and F_2 and therefore links the two firms. In AKM terms, W_1 is a mover. Worker W_2 stays at F_1 for two periods, while W_3 stays at F_2 for two periods. Both are stayers.

The diagonal blocks are count matrices. In this example, each worker is observed twice, each firm three times, and each year three times, so

$$G_{WW} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad G_{FF} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}, \quad G_{YY} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}. \quad (7)$$

The off-diagonal blocks are cross-tabulations between factors. The worker-firm block is

$$C_{WF} = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}. \quad (8)$$

The first row shows that worker W_1 appears once at firm F_1 and once at firm F_2 . Workers W_2 and W_3 are stayers, appearing twice at F_1 and F_2 , respectively. The worker-year block is

$$C_{WY} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}. \quad (9)$$

Each worker is observed once in each year. The firm-year block is

$$C_{FY} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}. \quad (10)$$

Firm F_1 appears twice in year Y_1 and once in year Y_2 ; firm F_2 has the opposite pattern. Together, the diagonal count blocks and off-diagonal cross-tabulation blocks form the full Gramian.

With column order $(W_1, W_2, W_3, F_1, F_2, Y_1, Y_2)$, the full Gramian is

$$G = \left(\begin{array}{ccc|ccc} 2 & 0 & 0 & 1 & 1 & 1 \\ 0 & 2 & 0 & 2 & 0 & 1 \\ 0 & 0 & 2 & 0 & 2 & 1 \\ \hline 1 & 2 & 0 & 3 & 0 & 2 \\ 1 & 0 & 2 & 0 & 3 & 1 \\ \hline 1 & 1 & 1 & 2 & 1 & 3 \\ 1 & 1 & 1 & 1 & 2 & 0 \end{array} \right). \quad (11)$$

The worker-firm submatrix represents the bipartite graph algebraically. The diagonal entries are worker and firm counts, and the entries of C_{WF} are edge multiplicities between workers and firms. Changing the sign of C_{WF} turns this submatrix into a graph Laplacian: its off-diagonal entries are non-positive, and every row sums to zero because each diagonal count cancels the off-diagonal spell counts in the same row. For a worker, the diagonal entry is the number of employment spells. The off-diagonal entries give the number of those spells at each firm. Firm rows have the analogous interpretation, with spell counts summed over workers.

$$L_{WF} = \left(\begin{array}{ccc|cc} 2 & 0 & 0 & -1 & -1 \\ 0 & 2 & 0 & -2 & 0 \\ 0 & 0 & 2 & 0 & -2 \\ \hline -1 & -2 & 0 & 3 & 0 \\ -1 & 0 & -2 & 0 & 3 \end{array} \right). \quad (12)$$

The same Laplacian construction applies to any pair of fixed effects. These pairwise Laplacians are the basis of the preconditioner in Section 6. MAP does not use them directly. It alternates among the diagonal worker, firm, and year blocks, while cross-factor information passes from one update to the next through the residual.

5. Alternating Projections and Graph Connectivity

The workhorse algorithm for multi-way fixed effects is the Method of Alternating Projections (MAP), also referred to as iterative demeaning or the “zig-zag” algorithm (Guimaraes and Portugal 2010; Gaure 2013). Many packages use MAP or a variant, often with acceleration methods (Berge 2018; Correia 2017). For example, `fixest` uses Irons-Tuck extrapolation (Irons and Tuck 1969; Bergé et al. 2026). The fixed-effect graph and its algebra, introduced in Sections 3 and 4, determine MAP’s convergence rate.

MAP solves the FWL residualization problem by iterating over one fixed effect at a time. In the worker-firm-year model, MAP first subtracts worker means from the current residual, then firm means from the updated residual, and then year means. It repeats this sequence until convergence.

We write the FWL normal equations (see Equation 4) in block form with $D = [D_W \quad D_F \quad D_Y]$ as

$$\begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix} \begin{pmatrix} \alpha_W \\ \alpha_F \\ \alpha_Y \end{pmatrix} = \begin{pmatrix} D_{W'} W \mu \\ D_{F'} W \mu \\ D_{Y'} W \mu \end{pmatrix}. \quad (13)$$

Equivalently,

$$G_{WW}\alpha_W + C_{WF}\alpha_F + C_{WY}\alpha_Y = D_{W'} W \mu, \quad (14)$$

$$C_{(WF)'}\alpha_W + G_{FF}\alpha_F + C_{FY}\alpha_Y = D_{F'} W \mu, \quad (15)$$

$$C_{(WY)'}\alpha_W + C_{(FY)'}\alpha_F + G_{YY}\alpha_Y = D_{Y'}W\mu. \quad (16)$$

Rearranging each equation expresses one block of effects conditional on the others. Substituting $C_{WF} = D_{W'}WD_F$ and $C_{WY} = D_{W'}WD_Y$ into the first equation and factoring out $D_{W'}W$ gives

$$G_{WW}\alpha_W = D_{W'}W(\mu - D_F\alpha_F - D_Y\alpha_Y). \quad (17)$$

Because G_{WW} is diagonal, with entries equal to workers' total observation weights, we obtain α_W by dividing each worker's weighted partial residual by that worker's total observation weight. For firms, the same rearrangement gives

$$G_{FF}\alpha_F = D_{F'}W(\mu - D_W\alpha_W - D_Y\alpha_Y), \quad (18)$$

and the year equation is analogous. Because G_{WW} , G_{FF} , and G_{YY} are all diagonal, we solve each equation by computing a weighted group mean in a single pass over observations.

MAP repeats these diagonal block solves. Holding the other effects fixed, it updates the worker effects from the current partial residual, then performs the same step for firms and years. A complete pass cycles through the fixed-effect dimensions and subtracts the weighted group mean of the current partial residual for each factor.

The cross-tabulation blocks C_{WF} , C_{WY} , and C_{FY} enter the algorithm only indirectly. For instance, the worker update is computed from the partial residual $\mu - D_F\alpha_F - D_Y\alpha_Y$, while the firm update is computed from $\mu - D_W\alpha_W - D_Y\alpha_Y$. MAP therefore does not solve the worker-firm, worker-year, and firm-year subproblems jointly. Instead, each block update changes the residual used by the next update. MAP converges quickly when the graph is well connected. In a high-mobility worker-firm panel, many workers move across firms, so that worker and firm effects can be compared through many overlapping employment histories. A high wage at one firm can then be related to wages earned by the same workers at other firms. A worker update quickly changes the information available to the next firm update, and vice versa.

When mobility is sparse, sorting is strong, or one factor is nearly nested in another, MAP slows down. Movers form the edges of the worker-firm graph and provide the information needed to separate worker effects from firm effects. MAP uses this information only indirectly, through repeated residual updates. When those edges are few or concentrated in narrow regions of the graph, repeated residual updates separate the effects only gradually. MAP still converges, but it may repeat these updates many times. A complete pass is cheap because the diagonal block solves reduce to one-pass group means.

We use the normalized-Laplacian gap $\lambda_{2,qr}$ as a diagnostic for MAP difficulty. Smaller values indicate weaker pairwise connectivity. For models with three or more fixed effects, the statistic describes one factor pair and is not a convergence bound for the full model. A faster solver must use the pairwise links directly instead of passing their information through one factor update at a time.

6. The Factor-Pair Schwarz Preconditioner

6.1. Preconditioners

LSMR can use the pairwise links through a preconditioner. MAP cannot take them as an input: its update rule is fully determined by the list of fixed-effect dimensions, and the cross-tabulations affect one update

only through the residual passed to the next. Iterative solvers such as LSMR accept an additional input, a preconditioner, that can incorporate the graph structure. We therefore replace factor-by-factor demeaning via MAP with LSMR (Fong and Saunders 2011), an iterative least-squares algorithm that improves an initial guess through repeated residual corrections. Switching solvers alone offers little gain: weakly connected designs also leave LSMR with components of G that take many iterations to remove. The speedup depends on a good preconditioner. The rest of this section develops one.

LSMR converges quickly when G is balanced enough that every component of the residual shrinks at a comparable rate. With weak links, sparse mobility, or near nesting, G becomes unbalanced: LSMR removes some components after a few iterations while others shrink much more slowly. This imbalance in G is what we mean by a poorly conditioned system. A preconditioner rescales the system so that LSMR reduces the easy and difficult components at more similar rates, without changing the least-squares solution.

The ideal preconditioner is G^{-1} .⁵ If $M^{-1} = G^{-1}$, then the matrix seen by LSMR is

$$M^{-1}G = G^{-1}G = I. \quad (19)$$

With this ideal preconditioner, LSMR would recover the solution after one correction because the inverse solves the system directly. Forming G^{-1} , however, would require solving the fixed-effect normal equations themselves. The ideal inverse is therefore a benchmark, not an implementable preconditioner. A useful preconditioner must approximate enough of G^{-1} to reduce the components that otherwise converge slowly. The time spent constructing and repeatedly applying it must be offset by the iterations it saves.⁶

6.2. From the Block Inverse to the Diagonal Preconditioner

The block structure of G^{-1} shows which parts of the ideal inverse a feasible preconditioner should retain.

For the worker-firm-year AKM model, the Gramian has the block form

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}, \quad (20)$$

with diagonal weighted-count blocks G_{WW}, G_{FF}, G_{YY} and off-diagonal cross-tabulations C_{WF}, C_{WY}, C_{FY} . The standard formula for inverting a block matrix shows how each off-diagonal block enters G^{-1} . For the two-factor block

$$G_2 = \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}, \quad (21)$$

it gives the explicit formula

$$G_2^{-1} = \begin{pmatrix} G_{WW}^{-1} + G_{WW}^{-1}C_{WF}S^{-1}C_{(WF)'}G_{WW}^{-1} & -G_{WW}^{-1}C_{WF}S^{-1} \\ -S^{-1}C_{(WF)'}G_{WW}^{-1} & S^{-1} \end{pmatrix}. \quad (22)$$

⁵As in any model with several fixed effects, the level effects are pinned down only up to a normalization: we can add a constant to every worker effect and subtract it from every firm effect without changing the fitted values $D\alpha$, and likewise for years. The dummy-coded $G = D'WD$ and the smaller blocks inverted below are therefore not invertible until this ambiguity is removed - one normalization for the worker-firm pair, and a second once year effects are added, as in the Section 4 example. We adopt the standard normalization within each connected set and read every inverse below as that of the resulting system. The estimated effects depend on the normalization; the residualized outcome and regressors, which are all the regression uses, do not.

⁶LSMR never constructs $M^{-1}G$ or G explicitly. It only multiplies vectors by D and D' and applies M^{-1} .

$$S = G_{FF} - C_{(WF)'} G_{WW}^{-1} C_{WF}. \quad (23)$$

The formula shows that every block of G_2^{-1} depends on C_{WF} only through S ; the cross-tabulation enters through matrix products, never as its own inverse. G_{WW} is diagonal, so G_{WW}^{-1} is a division by weighted worker counts. The matrix S is the Schur complement: it is the firm-side mobility system left after eliminating the worker effects. Solving this system is expensive for modern worker-firm register data. Exact factorization makes the matrix denser by creating additional nonzero entries, separate connected components require separate normalizations, and weak mobility makes some components slow to resolve. S^{-1} therefore accounts for almost all the cost of the solve. The same block-inverse calculation extends to three factors. The closed form has more terms, but every block of G^{-1} still depends jointly on the cross-tabulations C_{WF}, C_{WY}, C_{FY} .

The coarsest approximation to G^{-1} keeps only the diagonal count inverses and drops the Schur-complement corrections,

$$M_{\text{diag}}^{-1} = \text{diag}(G_{WW}^{-1}, G_{FF}^{-1}, G_{YY}^{-1}). \quad (24)$$

Applying this preconditioner requires a single division by weighted level counts. It is the diagonal preconditioner used with LSMR in `FixedEffectModels.jl` (Fong and Saunders 2011; `FixedEffectModels.jl` Developers 2026). Diagonal scaling records how many observations a level carries, but not how that level connects to the rest of the labor market. Level counts can already be useful when employment is concentrated in a few large firms, such as Novo Nordisk in Denmark or Samsung in South Korea, because the size differences alone remove an important source of scale variation. Diagonal scaling does not record whether workers at those firms link them broadly to other firms or remain concentrated in a narrow corner of the mobility graph.

6.3. The Factor-Pair Schwarz Approximation

Additive Schwarz preconditioning approximates a large problem by solving smaller, overlapping problems and adding their corrections (Xu 1992; Toselli and Widlund 2005). Here, the small problems retain the pairwise connectivity that diagonal scaling drops. In the AKM case, one problem contains workers and firms, another contains workers and years, and a third contains firms and years. The worker-firm problem moves residual information along observed employment links; the other two pair problems do the same for worker-year and firm-year links. We combine the three pair corrections in the full vector of fixed-effect coefficients. These corrections let LSMR account directly for the main pairwise links in the Gramian, while the outer iteration handles the remaining three-way coupling.

Consider the worker-firm pair. Its local problem is the two-factor block from the Schur calculation,

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}. \quad (25)$$

Figure 4 compares this worker-firm pair block with the single diagonal block solved by a factor-level MAP update.

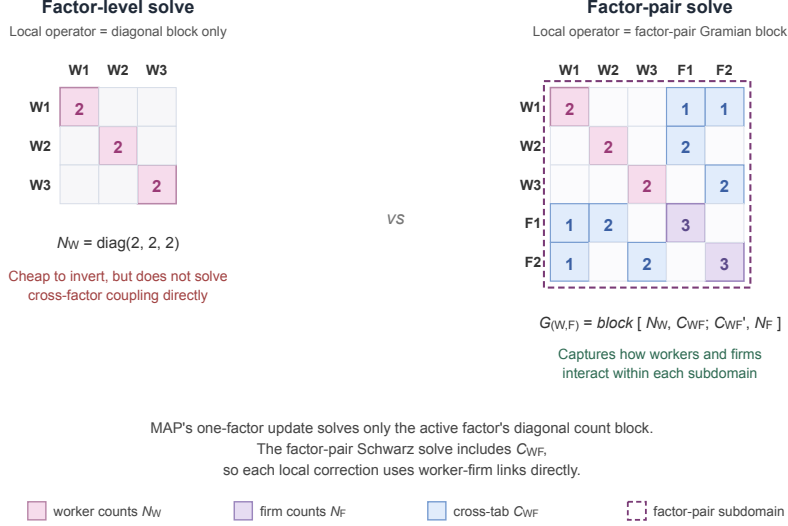


Figure 4: Matrices used by MAP and the factor-pair preconditioner in the example from Section 4. A MAP worker update (left) uses only the diagonal worker-count matrix $G_{WW} = \text{diag}(2, 2, 2)$. The factor-pair update (right) adds the firm-count matrix $G_{FF} = \text{diag}(3, 3)$ and the matrix of worker-firm observation counts C_{WF} . The dashed outline marks the worker and firm coefficients that are updated together.

Its inverse carries C_{WF} through the Schur complement, so the local correction retains the worker-firm mobility geometry that diagonal scaling discards. To use this correction in the three-factor problem, let R_{WF} select the worker and firm entries from the full coefficient vector $\alpha = [\alpha_W; \alpha_F; \alpha_Y]$; its transpose $R_{(WF)'}^T$ places the resulting correction back into the full vector. The diagonal matrix $\tilde{D}_{WF}$ weights levels that appear in more than one pair problem so their corrections are not counted twice. These entries are called partition-of-unity weights. The exact worker-firm contribution is

$$P_{WF}^{-1} = R_{(WF)'}^T \tilde{D}_{WF} \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}^{-1} \tilde{D}_{WF} R_{WF}. \quad (26)$$

The worker-year and firm-year contributions P_{WY}^{-1} and P_{FY}^{-1} are built the same way from G_{WW}, C_{WY}, G_{YY} and G_{FF}, C_{FY}, G_{YY} . With three factors each level appears in exactly two pair problems. Because the weights act on both sides of each pair contribution, we set them so that the squared weights on a shared level sum to one; a level in two pairs then carries $\frac{1}{\sqrt{2}}$. The exact factor-pair Schwarz preconditioner is the sum of these three contributions,

$$P^{-1} = P_{WF}^{-1} + P_{WY}^{-1} + P_{FY}^{-1}. \quad (27)$$

The three terms include the worker-firm, worker-year, and firm-year cross-tabulations separately. The simultaneous worker-firm-year coupling remains for the outer LSMR iteration, along with the error introduced by splitting shared levels across pairs.

6.4. Approximate Pair Solves via Graph Laplacians

For P^{-1} to serve as the operator M^{-1} inside LSMR, each pair contribution must be computed without solving a large dense system. We invert the pair block directly when a factor pair has few levels. In the example worker-firm panel of Section 4, the pair block has three worker levels and two firm levels;

after the normalization of Section 6.1 removes the one free constant, the local solve is a 4×4 inversion. Direct inversion is impractical at the scale of modern worker-firm register data, where a single pair can carry hundreds of thousands of levels per side. It becomes the same large linear-algebra problem that the preconditioner is meant to avoid. The pair block's graph-Laplacian structure provides an alternative.

For a worker-firm pair, the local pair step solves the pair-Gramian system

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix} x = u, \quad (28)$$

where u is the weighted worker-firm part of the current LSMR residual. This block is not a graph Laplacian because its off-diagonal entries C_{WF} are non-negative. Let $T_{WF} = \text{diag}(I_W, -I_F)$ flip the sign of the firm entries, so that $T_{WF}^2 = I$. Multiplying the pair block by T_{WF} on both sides gives

$$L_{WF} = T_{WF} \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix} T_{WF} = \begin{pmatrix} G_{WW} & -C_{WF} \\ -C_{(WF)'} & G_{FF} \end{pmatrix}, \quad (29)$$

a weighted bipartite graph Laplacian: symmetric, with non-positive off-diagonals and zero row sums. Because $T_{WF}^2 = I$, the pair-Gramian solve follows from the Laplacian solve by the same flip on each side,

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}^{-1} = T_{WF} L_{WF}^{-1} T_{WF}, \quad (30)$$

where both inverses are read as in Section 6.1. The solve fixes the free constant on each connected component by returning the zero-mean solution; the residualized variables do not depend on this choice. A single Laplacian solve therefore yields the pair-Gramian solution: we flip the residual, apply L_{WF}^{-1} , and flip back.

For preconditioning, this local solve need not be exact because the outer LSMR iteration refines the remaining error. We approximate the Laplacian solve using sparse approximate Cholesky factorizations (Spielman and Teng 2014; Gao et al. 2025). An exact Cholesky factorization gradually turns the sparse pair block into a denser matrix. Eliminating a worker creates new nonzero entries linking every pair of firms that worker visited, and these entries accumulate as more workers are eliminated. These extra entries are called fill-in; storing and updating them can make the exact factorization expensive. In the worst case, the calculation requires roughly k^3 operations and memory proportional to k^2 for a k -level system, as in a dense factorization. The randomized approximation controls this growth. Its cost is roughly proportional to the number of observed worker-firm links, with additional factors that grow only logarithmically with problem size. After transforming the approximate Laplacian solve back to worker-firm coordinates, we write A_{WF} for the resulting approximate pair-Gramian solve.

The same construction yields A_{WY} and A_{FY} for the other two pairs. Replacing the exact pair inverses in the Schwarz sum with these approximations gives the implemented preconditioner,

$$M^{-1} = \sum_{(q,r)} R_{(qr)'} \tilde{D}_{qr} A_{qr} \tilde{D}_{qr} R_{qr}. \quad (31)$$

In the worker-firm-year case each factor receives a diagonal contribution from its two pair subdomains and each off-diagonal correction from the corresponding pair.

The approximate pair inverse adds a second approximation to the pairwise splitting. The exact P^{-1} replaces the full three-factor inverse with exact pair solves; M^{-1} approximates each of those solves through A_{qr} . These two approximations affect only how LSMR chooses each update. The fitted residuals are unchanged, and LSMR still solves the original fixed-effect least-squares problem to the requested tolerance.

6.5. Implementation Strategy

Figure 5 summarizes the full construction. We divide the fixed-effect graph into overlapping factor pairs. For each pair, a sign flip turns the local Gramian block into a graph Laplacian. A sparse approximate Cholesky routine then approximates the pair inverse without letting the matrix become dense. We weight the overlapping corrections so that shared fixed-effect levels are not counted twice, then add them to form the Schwarz preconditioner M^{-1} .

The outer LSMR iteration uses this preconditioner to choose better-scaled updates (Fong and Saunders 2011; Arridge et al. 2014; Yang et al. 2024). Once constructed, the same preconditioner can be reused to residualize the outcome and every covariate. Appendix A gives the algorithmic details.

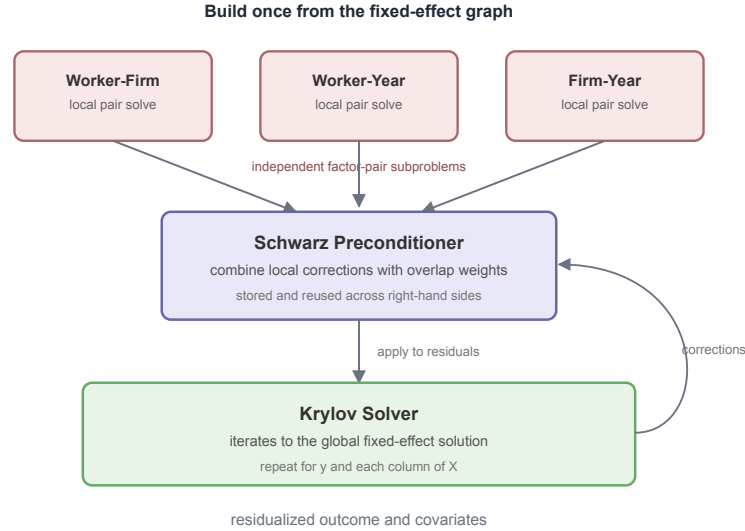


Figure 5: Construction of the factor-pair preconditioner. Each local problem combines two fixed-effect dimensions and retains their observed links. A sign change turns its matrix into a graph Laplacian, the standard matrix representation of a weighted graph. Sparse approximate Cholesky solves each local system while limiting the extra nonzero entries created during factorization, which saves memory and computation. The weighted sum of these local solutions serves as the preconditioner for LSMR.

7. Benchmarks

When connectivity is weak, MAP’s factor-by-factor updates pass information slowly through the fixed-effect graph. Factor-pair preconditioning should help most on these graphs, but may add unnecessary setup cost on well-connected ones. We test this prediction on controlled synthetic designs and public benchmark datasets.

The main runtime tables use package-level regression APIs, as does the public PyFixest benchmark suite, rather than timing the demeaning step alone. Each timing covers the full regression workflow: model setup, construction of the fixed-effect representation, residualization of the outcome and covariates, and

estimation of the coefficient of interest. Appendix C compares the coefficient estimates returned by the preconditioned solver and MAP. It also reports the memory cost of storing factor-pair information.

For cross-package comparisons, every software implementation receives the same input data but uses its own default rules for singleton fixed-effect levels (levels that occur only once) and observations removed because of separation. For successful fits, we record the retained-observation count; for failed attempts, we record the convergence status and error. Any shared control is stated with the relevant table. The comparisons are not constrained to a common estimation sample. The later single-package mechanism experiments use a common prepared sample because they isolate the preconditioner.

The runtime tables report λ_2 for the relevant factor pair. We compute it after removing singleton fixed-effect levels. For a disconnected pair graph, we compute λ_2 within each connected component and report the smallest value. The number in parentheses is that component’s share of retained observations. Pairwise λ_2 remains a diagnostic rather than a convergence bound for models with three or more fixed effects.

The OLS package-runtime tables compare six configurations: PyFixest MAP, PyFixest LSMR with no, diagonal, or factor-pair preconditioning, R `fixest`, and `FixedEffectModels.jl`:

Configuration	Package	Algorithm
<code>rust-map</code>	PyFixest	Vanilla Rust MAP, without acceleration.
<code>within-off</code>	PyFixest	LSMR without preconditioning.
<code>within-diagonal</code>	PyFixest	LSMR with diagonal preconditioning.
<code>within</code>	PyFixest	LSMR with factor-pair Schwarz preconditioning.
<code>fixest</code>	R <code>fixest</code>	Accelerated MAP with Irons-Tuck and other optimizations (Bergé et al. 2026).
<code>FEM.jl</code>	<code>FixedEffectModels.jl</code>	Diagonally preconditioned LSMR (Fong and Saunders 2011; FixedEffectModels.jl Developers 2026).

Reported CPU times are medians over each benchmark’s measured repetitions. Headline OLS uses the adaptive R1/R2/R3 rule; experiments with a fixed count use three repetitions. All runs use an Apple M4 Mac mini with 10 CPU cores and 16 GB of memory running macOS 15.3.1.

We do not include `reghdfe` (Correia 2014; 2017) directly in the benchmark tables because Stata is not open source and we lack a license. `reghdfe` is a mature accelerated-MAP implementation; algorithmically, it belongs to the same family as `fixest`’s accelerated MAP.

7.1. Runtime Benchmarks

The implementations use different rules for deciding when to stop, so their reported tolerance settings are not directly comparable. We first compare package defaults. The achieved-precision experiment later in this section compares every method using the same measures of coefficient and residual error.

The main OLS benchmarks use synthetic AKM-style panels with one million observations, one covariate, ten periods, and worker, firm, and year fixed effects. They vary the worker-firm graph in two ways while holding the rest of the data-generating process fixed. The mobility designs progressively reduce the number of workers who change firms. The sorting designs hold the move probability at one but assign a larger share of moves within groups of firms. Appendix B reports further synthetic designs, the

empirical HDFE benchmarks assembled by Sergio Correia, and the exact package-default AKM timings behind Figure 1.

7.1.1. Worker Mobility

Lower worker mobility leaves fewer workers connecting multiple firms. Because MAP updates one fixed-effect dimension at a time, information about firm effects moves more slowly through the iteration. The factor-pair preconditioner uses the worker-firm links directly, so its relative advantage grows as mobility falls. The top row of Figure 1 shows worker-firm λ_2 falling from 0.232 to 2.41×10^{-5} . Under package defaults, all configurations in the first two designs complete within 3.60 seconds. In the remaining designs, PyFixest MAP slows sharply or reaches its cap, and LSMR without preconditioning reaches its cap. Factor-pair LSMR falls from 0.543 seconds in the first design to 0.365 seconds in the last.

7.1.2. Sorting Among Movers

Every worker changes firms between adjacent periods in these designs. The sorting parameter ρ controls how strongly workers are matched to similar firms; larger values concentrate moves within groups and weaken the links between them. The bottom row of Figure 1 reports this comparison. Worker-firm λ_2 falls from 0.222 at $\rho = 0$ to 2.13×10^{-4} at $\rho = 150,000$. MAP rises from 0.368 seconds at $\rho = 0$ to 60.1 seconds at $\rho = 10,000$, then reaches its default cap at $\rho = 150,000$. LSMR without preconditioning reaches its default cap in the last four designs. Diagonal LSMR rises from 0.367 to 1.73 seconds. Factor-pair LSMR remains below 1.1 seconds and takes 0.651 seconds at $\rho = 150,000$; it is faster than diagonal LSMR in the three least-connected designs.

7.1.3. When Preconditioner Setup Dominates

The AKM designs vary connectivity gradually. The public fixest benchmark suite gives a sharper comparison between a dense, well-connected design and a sparse, nearly nested one (Bergé et al. 2026). Both contain 10 million observations, one covariate, and worker, firm, and year fixed effects.

Simple and difficult fixest designs (10 million observations, three fixed effects).

Design	λ_2 (share)	PyFixest MAP	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair	fixest	FEM.jl
simple (well-connected)	0.622 (1.00)	2.35s	2.72s	2.16s	11.2s	2.52s	2.08s
difficult (near-nested)	8.35×10^{-8} (1.00)	326.2s	10.2s	capped (0/3)	4.63s	62.2s	27.0s

Note: Times are in seconds and cover each OLS regression from model setup through coefficient estimation. Each package uses its default settings. A preliminary run sets the number of measured runs: 20 if it takes less than 1 second, 7 if it takes 1 to 10 seconds, and 3 if it takes longer. When only k of n planned runs return an estimate, $t(\frac{k}{n})$ reports their median time t . A dash means that no timing is available. capped (0/ n) means that no run finishes before the iteration limit; failed (0/ n) marks a failure other than reaching that limit. Both designs have 10 million observations, one covariate, and worker, firm, and year fixed effects. In the λ_2 (share) column, smaller values mean weaker worker-firm connectivity. We compute λ_2 after removing singleton fixed-effect levels. For a disconnected pair graph, we report the smallest component value; the number in parentheses is that component's share of retained observations.

On the dense design, all methods except factor-pair LSMR finish in 2.16 to 2.69 seconds; factor-pair LSMR takes 11.5 seconds because its setup cost is not recovered in one fit. On the near-nested design, factor-pair LSMR takes 4.45 seconds, compared with 11.0 seconds without preconditioning, 27.8 seconds for FEM.jl,

63.5 seconds for `fixest`, and 337.2 seconds for `PyFixest` MAP. `PyFixest` diagonal LSMR reaches its default cap on that design.

7.1.4. Iterations After Setup

The iteration-count diagnostic separates construction from subsequent solves and uses the 100,000-observation versions of the same simple and difficult designs.

Iterations on the simple and difficult designs (100,000 observations).

Design	map (passes)	off	diagonal	additive
simple	14	37	16	14
difficult	8159	249	180	22

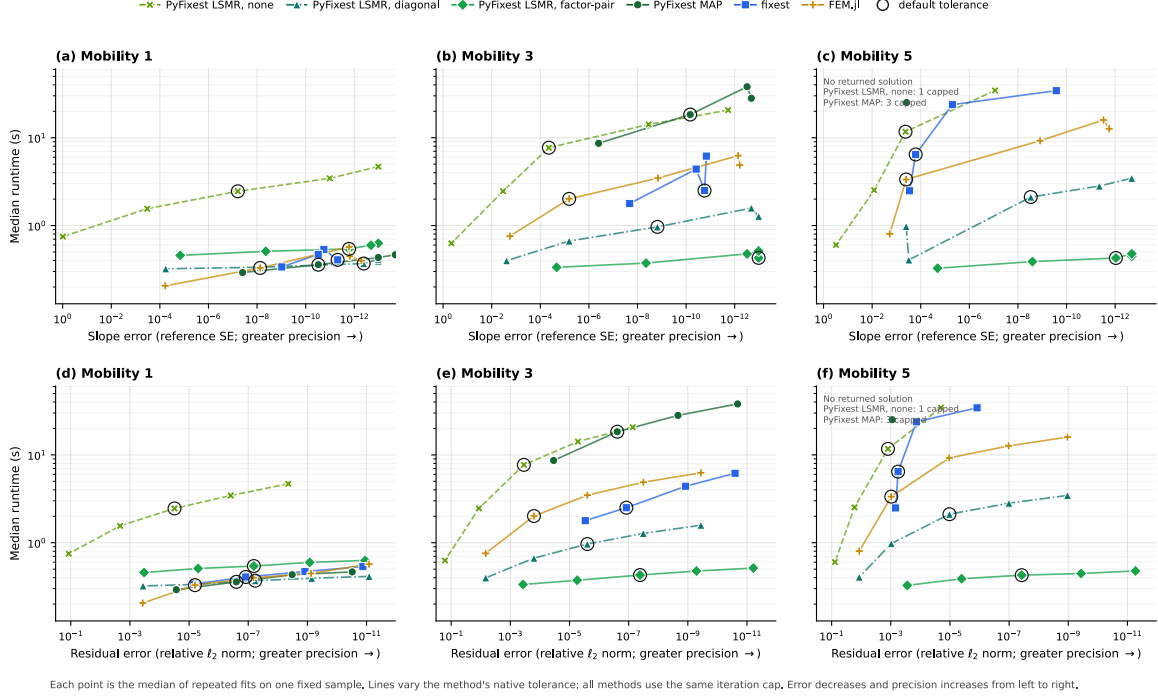
Note: The MAP column counts complete passes over all fixed-effect dimensions. The other columns count LSMR iterations: `off` uses no preconditioner, `diagonal` scales by fixed-effect group counts, and `additive` uses the factor-pair preconditioner. A MAP pass and an LSMR iteration perform different work, so their counts are not directly comparable. Each count is the median of three runs; each run removes the fixed effects from one outcome and one covariate.

Among the LSMR configurations, the additive preconditioner needs 14 iterations on the simple design and 22 on the difficult one. The diagonal count rises from 16 to 180, while unpreconditioned LSMR rises from 37 to 249. Once the additive preconditioner has been built, the difficult design requires only eight more LSMR iterations than the simple design. The preconditioned solve is short in both cases; setup makes within unattractive on the simple one-shot regression.

7.2. Runtime and Achieved Precision

The methods apply their stopping tolerances to different quantities. `PyFixest` MAP defaults to 10^{-6} and stops when every observation-level residual changes by less than that amount after a full pass over the fixed-effect dimensions. R's `fixest` also defaults to 10^{-6} , but applies the threshold to successive fixed-effect coefficient iterates. It accepts either a small absolute change or a small scaled relative change, and periodically makes the same comparison for the residual sum of squares. `FixedEffectModels.jl` passes its default tolerance of 10^{-6} as both LSMR stopping tolerances. `within` defaults to 10^{-8} and stops when either the estimated least-squares residual is at most 10^{-8} times the norm of the variable being residualized (the right-hand side), or a scaled measure of the remaining first-order-condition error falls below 10^{-8} . This second measure, called the normal-equation residual, is $\|A^T r\|_2 / (\|A\|_F \|r\|_2)$, using LSMR's running norm estimates (Fong and Saunders 2011).

Figure 6 compares the methods on a common accuracy scale. It uses mobility designs 1, 3, and 5. Before timing, we repeatedly remove fixed-effect levels that occur only once. We do this once for each one-million-observation design and pass the resulting rows to every method. We then vary each package's own tolerance, use a common 10,000-iteration cap, and time three fits at every setting. The tight reference uses factor-pair LSMR at tolerance 10^{-14} . The package defaults for the iteration cap are 10,000 for `PyFixest` MAP, `fixest`, and `FixedEffectModels.jl`, and 1,000 for `within`.



Each point is the median of repeated fits on one fixed sample. Lines vary the method's native tolerance; all methods use the same iteration cap. Error decreases and precision increases from left to right.

Figure 6: Elapsed time and achieved accuracy on three AKM mobility designs. Each marker is the median of three fits on the same sample. Before timing, we remove fixed-effect levels observed only once and use the remaining observations for every method. Each line connects results for one method across requested tolerances. The top row measures coefficient error as $|\hat{\beta} - \hat{\beta}^*|_{SE(\hat{\beta}^*)}$. The bottom row measures residual error as $\frac{\|r - r^*\|_2}{\|r^*\|_2}$. The reference coefficient $\hat{\beta}^*$ and residual r^* come from factor-pair LSMR at tolerance 10^{-14} . Error decreases from left to right. Circled markers use each package's default tolerance. Every fit is limited to 10,000 iterations. Settings that return no estimate are omitted and named in the annotations.

7.3. Amortizing the Preconditioner

The accuracy comparison separates solver performance from differences in stopping rules. Factor-pair LSMR also has a fixed setup cost. It constructs the pair blocks and their approximate factorizations before starting the LSMR iterations. With fixed observations, weights, and fixed-effect identifiers, these objects can be reused.

7.3.1. Setup Cost Across Connectivity

We time these two stages separately on the AKM mobility designs. The two-factor specification absorbs worker and firm effects; the three-factor specification also absorbs year effects. Each cell is the median of five standalone solves at one million observations, with the same outcome and covariate used in both specifications.

Factor-pair setup and solve time across AKM connectivity.

Move probability δ	λ_2 (share)	2 FE setup	2 FE solve	3 FE setup	3 FE solve
1	0.232 (1.00)	0.109s	0.291s	0.113s	0.209s
0.5	0.0392 (1.00)	0.115s	0.255s	0.118s	0.234s
0.05	1.84×10^{-3} (0.87)	0.032s	0.135s	0.034s	0.233s
0.01	5.35×10^{-4} (0.61)	0.027s	0.124s	0.031s	0.216s
0.005	5.50×10^{-5} (0.52)	0.025s	0.093s	0.031s	0.193s
0.001	2.41×10^{-5} (0.29)	0.028s	0.045s	0.032s	0.149s

Note: Times are in seconds. For each design and specification, the table gives the median setup and solve time from five runs with 1 million observations and an LSMR tolerance of 10^{-12} . The two-fixed-effect specification absorbs worker and firm effects; the three-fixed-effect specification also absorbs year effects. In the λ_2 (share) column, smaller values mean weaker worker-firm connectivity. We compute λ_2 after removing singleton fixed-effect levels. For a disconnected pair graph, we report the smallest component value; the number in parentheses is that component's share of retained observations.

Setup is cheaper in the low-mobility designs. With two fixed effects, median setup declines from 0.119 seconds at $\delta = 1$ to 0.067 seconds at $\delta = 0.001$; with three fixed effects, it falls from 0.123 to 0.073 seconds. Solve time falls as well, from 0.323 to 0.057 seconds with two effects and from 0.239 to 0.176 seconds with three. The low-mobility worker-firm graph has fewer cross-firm links to store and factorize. As a result, the additive runtime in Figure 1 can decline while MAP and diagonal LSMR slow down.

7.3.2. Ten Regressions on the Same Fixed Effects

Many empirical projects estimate several specifications on the same sample and fixed effects. The factor-pair objects can then be constructed once and reused. We measure ten sequential regressions on both one-million-observation `fixest` designs, using the same outcome and ten covariates for all three policies. One additive policy constructs a new preconditioner for every regression; the other keeps one preconditioner for all ten.

Ten regressions with rebuilt and cached preconditioners.

Design	Policy	Setup	Solve	Total	Speedup
simple	Diagonal	0.048s	0.553s	0.601s	1.0x
	Additive, rebuilt	2.80s	1.51s	4.29s	0.1x
	Additive, cached	0.281s	1.51s	1.79s	0.3x
difficult	Diagonal	0.049s	43.6s	43.6s	1.0x
	Additive, rebuilt	0.379s	1.20s	1.58s	27.5x
	Additive, cached	0.040s	1.26s	1.30s	33.6x

Note: For each policy, we sum setup and solve times over ten sequential regressions, then take the median across three repetitions. Times are in seconds. Each design has 1 million observations and worker, firm, and year fixed effects. Each regression removes the fixed effects from the common outcome and one of ten covariates at an LSMR tolerance of 10^{-12} . Speedup divides the diagonal total time by the reported total time. Additive, `rebuilt` constructs a new factor-pair preconditioner for each regression; Additive, `cached` reuses one preconditioner for all ten.

Even across ten regressions, the additive preconditioner is slower on the simple design. Diagonal preconditioning takes 0.692 seconds. Rebuilding the additive preconditioner takes 4.46 seconds, while caching lowers this to 1.83 seconds. Even after avoiding nine constructions, the cached additive path is still more than twice as slow as diagonal preconditioning on this well-connected graph.

On the difficult design, diagonal preconditioning takes 50.3 seconds for the ten regressions. Rebuilding the additive preconditioner lowers this to 1.77 seconds, and caching lowers it further to 1.23 seconds.

Caching reduces additive setup time from 0.434 to 0.040 seconds; the repeated solves take 1.34 seconds with rebuilding and 1.19 seconds with caching.

7.4. Poisson and Other GLMs

Faster fixed-effect solves matter especially when an estimator requires several of them, as in generalized linear models fit by iteratively reweighted least squares (IRLS). Each IRLS step fits a weighted least squares problem in which the response and covariates are demeaned against the fixed effects (Correia et al. 2020; Stammann 2018). A single GLM fit therefore calls the demeaning routine repeatedly. The mapping from observations to fixed-effect levels stays the same, but the weights change between calls. This pattern arises in `ppmlhdfe`, Poisson fixed-effect estimators, and other IRLS-based GLM implementations.

PyFixest currently reuses the preconditioner built during the first weighted demeaning call. This avoids a new factorization at every IRLS step, but later problems use a preconditioner based on earlier weights. Reuse saves construction time and may require more LSMR iterations. Rebuilding may reduce the iteration count but incurs construction cost at every step. The table reports the current PyFixest reuse behavior.

The PPML comparison uses the simple and difficult designs from the `fixest` benchmark suite (Bergé et al. 2026) at $n = 1\text{M}$ observations, with one covariate and worker, firm, and year fixed effects. The compared implementations are R `fixest`’s `fepois`, `GLFixedEffectModels.jl`, and two PyFixest `fepois` paths: the default unpreconditioned `rust-map` and `within` with PyFixest’s current reuse policy. Packages apply their default rules for separated observations, and each implementation receives the same 100-iteration outer IRLS limit.

Poisson benchmarks (1 million observations, one covariate).

Design	FE	PyFixest MAP	fixest	GLFEM.jl	PyFixest LSMR factor-pair
simple (well-connected)	3	7.95s	4.63s	5.71s	9.74s
difficult (near-nested)	3	capped (0/3)	439.3s	129.8s	5.52s

Note: Times are in seconds for Poisson fixed-effect regressions with 1 million observations, one covariate, and worker, firm, and year fixed effects. `fixest` is R `fixest::fepois`; `rust-map` and `within` use PyFixest `fepois`; and `GLFEM.jl` is `GLFixedEffectModels.jl`. The `within` configuration reuses the first factor-pair preconditioner as the weights change across iteratively reweighted least squares steps. Each package applies its default rule for removing separated observations, where regressors and fixed effects perfectly predict some zero outcomes. If only k of the three planned fits return an estimate, $t(\frac{k}{3})$ gives their median time t . `capped (0/3)` means that no fit finishes within 100 iteratively reweighted least squares steps; `failed (0/3)` marks a failure other than reaching that limit.

On the simple design, all four paths finish in under ten seconds. `fixest` takes 4.72 seconds, `GLFEM.jl` 5.76, `rust-map` 7.86, and `within` 9.25. Runtime differences are much larger on the difficult design. `rust-map` does not converge within the iteration cap, `GLFEM.jl` takes 129.8 seconds, and `fixest` takes 439.4 seconds. `within` finishes in 5.43 seconds. Sparse worker-firm coupling slows MAP, while the factor-pair blocks encode those links directly.

8. Software

The open-source `within` project provides the solver benchmarked in Section 7 (within Developers 2026). Its computational core is written in Rust, with APIs for Python and R.

Interface	Package	Registry	Install
Rust	within	crates.io	cargo add within
Python	within-py	PyPI	pip install within-py
R	withinr	py-econometrics R-universe	install.packages("withinr", repos = "https://py-econometrics.r-universe. dev")

The Rust crate exposes the lower-level solver and its configuration types. The Python and R packages provide `solve` and `solve_batch` APIs that residualize one or several variables. PyFixest also provides the algorithm as a demeaning option (PyFixest Developers 2026).

The Python example below demeans an outcome and two covariates against worker and firm fixed effects, then runs the FWL regression on the demeaned variables:

```
import numpy as np
from within import solve_batch
# Worker and firm identifiers as a column-major uint32 array
n = 100_000
categories = np.asfortranarray(np.column_stack([
    np.random.randint(0, 5_000, n).astype(np.uint32),
    np.random.randint(0, 500, n).astype(np.uint32),
]))
# Outcome and covariates
beta = np.array([1.0, -2.0])
X = np.random.randn(n, 2)
y = X @ beta + np.random.randn(n)
# Residualize y and X jointly; the preconditioner is reused across columns
res = solve_batch(categories, np.column_stack([y, X]))
y_tilde, X_tilde = res.demeaned[:, 0], res.demeaned[:, 1:]
# FWL on the demeaned variables
beta_hat = np.linalg.lstsq(X_tilde, y_tilde, rcond=None)[0]
```

`solve_batch` follows the FWL workflow from Section 2: it residualizes the outcome and covariates together and reuses one factor-pair preconditioner across columns. Users do not need to construct the Gramian or its pairwise blocks themselves.

9. Conclusion

Graph connectivity links the econometric structure of a fixed-effect model to its computational cost. The benchmarks show that it also changes solver rankings. On dense, well-connected graphs, MAP is difficult to outperform because each pass is cheap, while constructing the factor pairs adds overhead. When mobility is low, sorting is strong, or effects are nearly nested, information passes slowly between updates under MAP and the factor-pair preconditioner is often much faster.

Constructing the factor pairs becomes cheaper in the low-mobility AKM designs. Its cost is incurred once for regressions that share the same sample, fixed effects, and weights. Caching lowers additive runtime in both simple and difficult designs, but only the difficult design is faster than diagonal preconditioning. For that design, caching reduces additive setup from 0.434 to 0.040 seconds and total time from 1.77 to 1.23 seconds.

PPML calls the demeaning routine repeatedly because each IRLS step solves a new weighted problem. In our benchmark, PyFixest reuses the first preconditioner as the weights change. Our results support factor-pair preconditioning when pairwise λ_2 is small, a fit is unexpectedly slow, or an estimator requires several fixed-effect solves.

Appendix A: Factor-Pair Schwarz Algorithm

Algorithm 1 implements the construction shown in Figure 5.

Algorithm 1. Factor-Pair Schwarz Preconditioner

Inputs

- Observation-level factor codes for Q fixed-effect dimensions.
- Diagonal weights W and local solver settings.
- Current LSMR residual r , indexed by the fixed-effect coefficients.

Preconditioner setup

- Enumerate all unordered factor pairs (q, r) with $q < r$.
- For each pair, build weighted count blocks G_{qq} , G_{rr} and the weighted cross-tabulation C_{qr} .
- Split the induced bipartite graph into connected components and create one local pair problem (a Schwarz subdomain) s per component.
- If fixed-effect level j appears in c_j subdomains, store the partition weight $\omega_j = \frac{1}{\sqrt{c_j}}$.
- For each subdomain, sign-flip one side to obtain a local Laplacian, remove the constant part that cannot be identified within the component from the local right-hand side, and build a zero-mean solve.
- Use a Schur-complement local solver: small reduced systems are solved directly, while larger reduced symmetric diagonally dominant (SDD) or Laplacian systems are solved with randomized approximate Cholesky.

Application during LSMR

- Initialize $z = 0$.
- For each subdomain s , form $h_s = \tilde{D}_s R_s r$.
- Compute the approximate local correction $u_s \approx A_s h_s$ on the zero-mean subspace.
- Accumulate $z \leftarrow z + R_{s'} \tilde{D}_s u_s$.
- Return $z = M^{-1}r$.

Appendix B: Additional Benchmark Results

We report the detailed AKM timings behind Figure 1 and results for synthetic and real data from the Correia collection. The script `scripts/paper_results.py` generates every table in Appendices B and C from recorded benchmark output; none of the numbers are entered by hand.

Controlled AKM Benchmarks

Panel A reports the exact package-default timings used in the left column of Figure 1. Panel B reports the other package-default PyFixest LSMR configurations, so its factor-pair column repeats Panel A’s PyFixest factor-pair result.

Worker Mobility

Mobility benchmark: package defaults (Panel A).

Move probability δ	λ_2 (share)	PyFixest MAP	fixest	FEM.jl	PyFixest LSMR factor-pair
1	0.232 (1.00)	0.405s	0.485s	0.335s	0.554s
0.5	0.0392 (1.00)	1.19s	0.894s	0.594s	0.542s
0.05	1.84×10^{-3} (0.87)	17.9s	2.59s	2.03s	0.428s
0.01	5.35×10^{-4} (0.61)	58.0s	5.86s	2.94s	0.409s
0.005	5.50×10^{-5} (0.52)	capped (0/3)	6.54s	3.38s	0.403s
0.001	2.41×10^{-5} (0.29)	capped (0/3)	7.25s	4.29s	0.373s

Note: Times are in seconds and cover each regression from model setup through coefficient estimation. Each package uses its default settings. Each design has 1 million observations, one covariate, and worker, firm, and year fixed effects. Move probability is the probability that a worker changes firms between adjacent periods. In the λ_2 (share) column, smaller values mean weaker worker-firm connectivity. We compute λ_2 after removing singleton fixed-effect levels. For a disconnected pair graph, we report the smallest component value; the number in parentheses is that component’s share of retained observations. If only k of the n planned runs return an estimate, $t(\frac{k}{n})$ gives their median time t . A dash means that no timing is available. capped (0/ n) means that no run finishes before the 10,000-iteration limit; Figure 1 plots the median elapsed time as a lower bound. failed (0/ n) marks a failure other than reaching that limit.

Mobility benchmark: PyFixest LSMR configurations (Panel B).

Move probability δ	λ_2 (share)	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair
1	0.232 (1.00)	2.48s	0.371s	0.554s
0.5	0.0392 (1.00)	3.65s	0.466s	0.542s
0.05	1.84×10^{-3} (0.87)	capped (0/3)	0.957s	0.428s
0.01	5.35×10^{-4} (0.61)	capped (0/3)	1.34s	0.409s
0.005	5.50×10^{-5} (0.52)	capped (0/3)	2.18s	0.403s
0.001	2.41×10^{-5} (0.29)	capped (0/3)	3.16s	0.373s

Note: Times are in seconds and cover each regression from model setup through coefficient estimation. Each mobility design has 1 million observations, one covariate, and worker, firm, and year fixed effects. Move probability is the probability that a worker changes firms between adjacent periods. The columns show PyFixest LSMR with no preconditioner, with diagonal scaling by fixed-effect group counts, and with factor-pair preconditioning. The factor-pair column repeats Panel A. In the λ_2 (share) column, smaller values mean weaker worker-firm connectivity. We compute λ_2 after removing singleton fixed-effect levels. For a disconnected pair graph, we report the smallest component value; the number in parentheses is that component’s share of retained observations. capped (0/ n) means that no run finishes before the 10,000-iteration limit.

Sorting Among Movers

Sorting benchmark: package defaults (Panel A).

Sorting strength ρ	λ_2 (share)	PyFixest MAP	fixest	FEM.jl	PyFixest LSMR factor-pair
0	0.222 (1.00)	0.411s	0.448s	0.325s	0.557s
20	0.0333 (1.00)	1.35s	0.857s	0.525s	0.950s
500	1.49×10^{-3} (1.00)	18.3s	1.68s	1.08s	0.804s
2000	6.55×10^{-4} (1.00)	35.6s	2.90s	1.39s	0.787s
10000	4.12×10^{-4} (1.00)	53.9s	5.59s	1.86s	0.727s
150000	2.13×10^{-4} (1.00)	capped (0/3)	6.16s	2.23s	0.623s

Note: Times are in seconds and cover each regression from model setup through coefficient estimation. Each package uses its default settings. Each design has 1 million observations, one covariate, and worker, firm, and year fixed effects. Every worker changes firms between adjacent periods; the parameter ρ controls how strongly workers sort across firms. In the λ_2 (share) column, smaller values mean weaker worker-firm connectivity. We compute λ_2 after removing singleton fixed-effect levels. For a disconnected pair graph, we report the smallest component value; the number in parentheses is that component's share of retained observations. If only k of the n planned runs return an estimate, $t(\frac{k}{n})$ gives their median time t . A dash means that no timing is available. capped (0/n) means that no run finishes before the 10,000-iteration limit; Figure 1 plots the median elapsed time as a lower bound. failed (0/n) marks a failure other than reaching that limit.

Sorting benchmark: PyFixest LSMR configurations (Panel B).

Sorting strength ρ	λ_2 (share)	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair
0	0.222 (1.00)	2.60s	0.362s	0.557s
20	0.0333 (1.00)	2.04s	0.427s	0.950s
500	1.49×10^{-3} (1.00)	capped (0/3)	0.814s	0.804s
2000	6.55×10^{-4} (1.00)	capped (0/3)	1.05s	0.787s
10000	4.12×10^{-4} (1.00)	capped (0/3)	1.35s	0.727s
150000	2.13×10^{-4} (1.00)	capped (0/3)	1.58s	0.623s

Note: Times are in seconds and cover each regression from model setup through coefficient estimation. Each sorting design has 1 million observations, one covariate, and worker, firm, and year fixed effects. Every worker changes firms between adjacent periods; the parameter ρ controls how strongly workers sort across firms. The columns show PyFixest LSMR with no preconditioner, with diagonal scaling by fixed-effect group counts, and with factor-pair preconditioning. The factor-pair column repeats Panel A. In the λ_2 (share) column, smaller values mean weaker worker-firm connectivity. We compute λ_2 after removing singleton fixed-effect levels. For a disconnected pair graph, we report the smallest component value; the number in parentheses is that component's share of retained observations. capped (0/n) means that no run finishes before the 10,000-iteration limit.

Correia Benchmark Collection

Synthetic Designs

The controlled AKM benchmarks in Section 7 vary mobility and sorting directly. The public and reproducible Correia HDFE benchmark collection covers a broader set of graph shapes: complete bipartite matching, uniform random matching with varying degrees of connectivity, assortative matching, and a small path-like design. It provides an independent reference set for comparing the same software implementations outside the AKM generator.

Correia synthetic benchmarks.

Dataset	λ_2 (share)	PyFixest MAP	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair	fixest	FEM.jl
synthetic-complete	1.00 (1.00)	0.109s	0.114s	0.128s	0.218s	0.087s	0.041s
synthetic-uniform-easy	0.409 (1.00)	0.160s	0.284s	0.188s	0.194s	0.165s	0.056s
synthetic-uniform-hard	0.0967 (1.00)	0.337s	0.437s	0.348s	0.970s	0.358s	0.233s
synthetic-uniform-harder	0.0125 (1.00)	0.757s	0.557s	0.474s	0.553s	1.18s	0.289s
synthetic-assortative	6.65×10^{-4} (0.70)	13.0s	1.25s	1.03s	0.382s	1.37s	1.24s

Note: Times are in seconds for OLS regressions using each package’s defaults. We planned three runs per cell. Each package applies its own default treatment of fixed-effect levels observed only once. In the λ_2 (share) column, smaller values mean weaker connectivity between id1 and id2. We compute λ_2 after removing singleton fixed-effect levels. For a disconnected pair graph, we report the smallest component value; the number in parentheses is that component’s share of retained observations. If only k of the three planned runs return an estimate, $t(\frac{k}{3})$ gives their median time t . A dash means that no timing is available. capped (0/3) means that no run finishes before the iteration limit; failed (0/3) marks a failure other than reaching that limit. synthetic-zigzag is omitted because the default MAP implementations reach the 10,000-iteration limit.

The synthetic collection covers complete, uniform, assortative, and path-like matching patterns. The no-preconditioner and diagonal LSMR results will be added once the package-default Correia run is complete; the unfilled cells are not used for rankings.

Real Data

The Correia collection also includes real benchmark data. Synthetic data sets match the overall shape of empirical co-occurrence graphs but smooth away several irregularities that often drive runtime. These include a few units that appear far more often than the rest, thin connections between otherwise dense groups, many small disconnected pieces, and interactions between identifiers that go beyond a single two-way pair. The empirical datasets contain these features. They therefore test the solvers in conditions that controlled data-generating processes only approximate.

Correia real-data benchmarks.

Dataset	λ_2 (share)	PyFixest MAP	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair	fixest	FEM.jl
credit	0.227 (1.00)	0.193s	0.464s	0.206s	0.242s	0.154s	0.097s
soccer	0.667 (1.00)	0.026s	0.056s	0.032s	0.065s	0.014s	0.010s

Dataset	λ_2 (share)	PyFixest MAP	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair	fixest	FEM.jl
enron	3.53×10^{-3} (0.99)	2.95s	1.58s	0.376s	0.402s	0.535s	0.317s
github	3.15×10^{-4} (0.32)	capped (0/3)	capped (0/3)	0.906s	0.310s	1.43s	1.44s
patents	2.59×10^{-4} (0.95)	capped (0/3)	1.47s	1.03s	0.388s	1.05s	0.576s
workers	1.37×10^{-4} (0.63)	capped (0/3)	capped (0/3)	1.28s	0.279s	2.00s	1.45s
schools	1.11×10^{-3} (1.00)	5.98s	1.59s	0.606s	0.266s	0.828s	0.487s
directors	2.56×10^{-4} (0.30)	capped (0/3)	1.49s	1.26s	0.327s	0.283s	0.462s

Note: Times are in seconds for OLS regressions using each package’s defaults. We planned three runs per cell. Each package applies its own default treatment of fixed-effect levels observed only once. In the λ_2 (share) column, smaller values mean weaker connectivity between id1 and id2. We compute λ_2 after removing singleton fixed-effect levels. For a disconnected pair graph, we report the smallest component value; the number in parentheses is that component’s share of retained observations. If only k of the three planned runs return an estimate, $t(\frac{k}{3})$ gives their median time t . A dash means that no timing is available. capped (0/3) means that no run finishes before the iteration limit; failed (0/3) marks a failure other than reaching that limit. A small λ_2 in a small component, as for directors, need not make the full sample difficult for MAP.

No single pairwise statistic captures all the irregular components in the real-data collection. The directors component attaining the smallest reported λ_2 covers 30 percent of the observations. The no-preconditioner and diagonal LSMR columns await the package-default Correia run. Pairwise λ_2 remains a diagnostic and does not serve as a selection rule.

Appendix C: Coefficient Estimates and Memory Use

Comparing Coefficient Estimates

We compare the coefficient estimates because MAP and LSMR use different rules for deciding when to stop. Even when both methods are implemented correctly, their estimates need only agree within the requested tolerances.

We use the 100K-observation versions of the simple and difficult `fixest` benchmark designs. All methods should agree closely on the simple design. In the difficult design, some remaining errors shrink much more slowly, so small differences in stopping rules are more likely to appear. Worker-firm λ_2 is approximately 0.622 in the simple design and 6.50×10^{-4} in the difficult design. The table compares estimates of one regression coefficient across four implementations.

Coefficient estimates across implementations (100,000 observations, three fixed effects, one covariate).

Design	Configuration	$\hat{\beta}_1$	Absolute difference
simple	PyFixest	1.00463872	-
	MAP		
	PyFixest	1.00463872	0
	LSMR		
	factor-pair		
	fixest	1.00463872	1.1×10^{-15}
difficult	FEM.jl	1.00463872	2.8×10^{-14}
	PyFixest	0.99953359	-
	MAP		
	PyFixest	0.99953387	2.8×10^{-7}
	LSMR		
	factor-pair		
	fixest	0.99953387	2.8×10^{-7}
	FEM.jl	0.99953385	2.6×10^{-7}

Note: Each row comes from one OLS regression fitted with the package’s default settings. $\hat{\beta}_1$ is the slope coefficient on `x1`. The final column reports its absolute difference from the PyFixest MAP estimate. The dash in the PyFixest MAP row marks this reference estimate. within is PyFixest LSMR with factor-pair preconditioning.

On the simple design, the largest coefficient difference is 2.8×10^{-14} . On the difficult design it is 2.8×10^{-7} . The packages use different criteria to decide when their iterative algorithms have converged. Small differences between their estimates are therefore expected on the difficult design.

Memory Use

MAP uses little memory: a complete pass needs only the current residuals and per-level group sums. A factor-pair preconditioner, by contrast, must retain the pair structure between iterations. We therefore measure how much additional memory factor-pair LSMR requires relative to MAP.

We record peak resident set size (peak RSS), the largest amount of physical memory used by the process during a run, on the simple and difficult fixed-effect designs. The main storage terms are the data matrix, the fixed-effect encodings, and the reusable factor-pair objects. We do not compare `fixest` and `FixedEffectModels.jl` here because peak RSS across languages also reflects R and Julia runtime overhead, data-loading choices, garbage collection, and package internals. Instead, we compare the preconditioned Rust implementation with Rust MAP inside the same Python package. The surrounding

regression code is shared; only the demeaning strategy differs. Both implementations run in isolated processes and report peak RSS through `ru_maxrss`.

Memory footprint (three fixed effects, one covariate).

Design	λ_2 (share)	PyFixest MAP	PyFixest LSMR factor-pair
<i>100K observations</i>			
simple (well-connected)	0.622 (1.00)	354 MiB	411 MiB
difficult (near-nested)	8.35×10^{-6} (1.00)	357 MiB	383 MiB
<i>1M observations</i>			
simple (well-connected)	0.622 (1.00)	758 MiB	1,000 MiB
difficult (near-nested)	8.35×10^{-6} (1.00)	775 MiB	953 MiB

Note: For each regression, we record the largest amount of physical memory used by an isolated Python process, in MiB (2^{20} bytes). The table compares PyFixest OLS regressions using MAP or factor-pair LSMR. Each regression has one covariate and worker, firm, and year fixed effects. The results cover the simple and difficult designs at 100,000 and 1 million observations; they do not show how memory changes with graph structure. In the λ_2 (share) column, smaller values mean weaker worker-firm connectivity. We compute λ_2 after removing singleton fixed-effect levels. For a disconnected pair graph, we report the smallest component value; the number in parentheses is that component's share of retained observations.

At 100K observations, the preconditioner adds 26-57 MiB. At 1M observations the overhead is 178-242 MiB, but it remains modest relative to the full panel data footprint. The additional storage holds factor-pair co-occurrences, partition weights, and local approximate Cholesky factors.

References

- Abowd, John M., Francis Kramarz, and David N. Margolis. 1999. "High Wage Workers and High Wage Firms." *Econometrica* 67 (2): 251–333. <https://doi.org/10.1111/1468-0262.00020>.
- Arridge, Simon R., Marta M. Betcke, and Lauri Harhanen. 2014. "Iterated Preconditioned LSQR Method for Inverse Problems on Unstructured Grids." *Inverse Problems* 30 (7): 75009. <https://doi.org/10.1088/0266-5611/30/7/075009>.
- Berge, Laurent. 2018. *Efficient Estimation of Maximum Likelihood Models with Multiple Fixed-Effects: The R Package Fenmlm*. Technical Report Nos. 2018–13.
- Bergé, Laurent R., Kyle Butts, and Grant McDermott. 2026. "Fixest: A Fast and Feature-Rich Framework for Econometric Estimations in R." <https://doi.org/10.48550/arXiv.2601.21749>.
- Correia, Sergio. 2014. "Reghdfe: Stata Module to Perform Linear or Instrumental-Variable Regression Absorbing Any Number of High-Dimensional Fixed Effects." <https://github.com/sergiocorreia/reghdfe>.
- Correia, Sergio. 2017. *Linear Models with High-Dimensional Fixed Effects: An Efficient and Feasible Estimator*. Technical report. <https://hdl.handle.net/10.2139/ssrn.3129010>.
- Correia, Sergio, Paulo Guimarães, and Tom Zylkin. 2020. "Fast Poisson Estimation with High-Dimensional Fixed Effects." *The Stata Journal* 20 (1): 95–115. <https://doi.org/10.1177/1536867X20909691>.
- FixedEffectModels.jl Developers. 2026. "Fixedeffectmodels.jl: Fast Estimation of Linear Models with High Dimensional Categorical Variables." <https://github.com/FixedEffects/FixedEffectModels.jl>.
- Fong, David Chin-Lung, and Michael Saunders. 2011. "LSMR: An Iterative Algorithm for Sparse Least-Squares Problems." *SIAM Journal on Scientific Computing* 33 (5): 2950–71. <https://doi.org/10.1137/10079687X>.
- Frisch, Ragnar, and Frederick V. Waugh. 1933. "Partial Time Regressions as Compared with Individual Trends." *Econometrica* 1 (4): 387–401. <https://doi.org/10.2307/1907330>.
- Gao, Yuan, Rasmus Kyng, and Daniel A. Spielman. 2025. "AC(k): Robust Solution of Laplacian Equations by Randomized Approximate Cholesky Factorization." *SIAM Journal on Scientific Computing*. <https://rasmuskyng.com/papers/GKS25.pdf>.
- Gaure, Simen. 2013. "OLS with Multiple High Dimensional Category Variables." *Computational Statistics & Data Analysis* 66 : 8–18. <https://doi.org/10.1016/j.csda.2013.03.024>.
- Goldsmith-Pinkham, Paul. 2026. *Tracking the Credibility Revolution across Fields*. Technical report.
- Guimaraes, Paulo, and Pedro Portugal. 2010. "A Simple Feasible Procedure to Fit Models with High-Dimensional Fixed Effects." *The Stata Journal* 10 (4): 628–49. <https://doi.org/10.1177/1536867X1101000406>.
- Irons, Bruce M., and Richard C. Tuck. 1969. "A Version of the Aitken Accelerator for Computer Iteration." *International Journal for Numerical Methods in Engineering* 1 (3): 275–77. <https://doi.org/10.1002/nme.1620010306>.
- Jochmans, Koen, and Martin Weidner. 2019. "Fixed-Effect Regressions on Network Data." *Econometrica* 87 (5): 1543–60. <https://doi.org/10.3982/ECTA14605>.

- Lovell, Michael C. 1963. "Seasonal Adjustment of Economic Time Series and Multiple Regression Analysis." *Journal of the American Statistical Association* 58 (304): 993–1010. <https://doi.org/10.1080/01621459.1963.10480682>.
- PyFixest Developers. 2026. "Pyfixest: Fast High-Dimensional Fixed Effects Regression in Python." <https://github.com/py-econometrics/pyfixest>.
- Spielman, Daniel A., and Shang-Hua Teng. 2014. "Nearly Linear Time Algorithms for Preconditioning and Solving Symmetric, Diagonally Dominant Linear Systems." *SIAM Journal on Matrix Analysis and Applications* 35 (3): 835–85. <https://doi.org/10.1137/090771430>.
- Stammann, Amrei. 2018. "Fast and Feasible Estimation of Generalized Linear Models with High-Dimensional K-Way Fixed Effects." <https://doi.org/10.48550/arXiv.1707.01815>.
- Toselli, Andrea, and Olof Widlund. 2005. *Domain Decomposition Methods: Algorithms and Theory*. Springer. <https://doi.org/10.1007/b137868>.
- within Developers. 2026. "Within: High-Performance Fixed-Effects Solver for Econometric Panel Data." <https://github.com/py-econometrics/within>.
- Xu, Jinchao. 1992. "Iterative Methods by Space Decomposition and Subspace Correction." *SIAM Review* 34 (4): 581–613. <https://doi.org/10.1137/1034116>.
- Yang, Mei, Gul Karaduman, and Ren-Cang Li. 2024. "Flexible Modified LSMR for Least Squares Problems."